NATURAL LANGUAGE PROCESSING

Project 1
Duplicate Question Detection

A complete beginner-to-implementation learning manual

Sentence Embeddings | Vector Representations | Cosine Similarity | Semantic Search

Prepared for students and faculty

Development environment: Windows | VS Code | Python 3.12 virtual environment

Prepared: 13 November 2026

About this manual

This manual documents Project 1, Duplicate Question Detection, from the first concepts through a working command-line implementation. It combines the supplied NLP Projects brief, the supplied environment setup guide, the exact code distributed for this project, and the terminal troubleshooting steps observed during the setup session.

Scope and execution status

The local command-line detector successfully loaded the model, prepared embeddings for eight reference questions, and reached the "Enter a question:" prompt. The actual result of a test query had not yet been shared when these notes were prepared. Therefore, this manual does not invent similarity scores or claim that a particular query has already produced a measured result.

Who should use this manual?

- Students: read the concepts first, execute the commands, then modify the questions and threshold as exercises.
- Faculty: use the explanations, terminal log, teaching notes, code walkthrough, troubleshooting section, and viva questions to conduct a practical session.

Learning outcomes

- Explain the difference between exact text matching and semantic similarity.
- Describe sentence embeddings as numerical vector representations of text.
- Load the all-MiniLM-L6-v2 sentence-transformer model.
- Generate embeddings for stored questions and for a new query.
- Calculate cosine similarity, rank candidates, and apply a threshold.
- Run and troubleshoot the program in VS Code using a Python virtual environment.
- Describe how the local application can be extended using FastAPI and PostgreSQL with pgvector.

Contents

- 1. Project overview and problem statement
- 2. Concepts: embeddings, vectors, cosine similarity, threshold and ranking
- 3. Development environment and folder structure
- 4. VS Code terminal execution: exact commands followed
- 5. Complete core program: app.py
- 6. Code walkthrough: statement-by-statement concepts
- 7. Testing the detector and interpreting output
- 8. Troubleshooting log from the actual setup session
- 9. Classroom exercises, evaluation and viva questions
- 10. Optional extension A: FastAPI service
- 11. Optional extension B: PostgreSQL and pgvector
- Appendices: complete API, database Python and SQL code

1. Project overview and problem statement

Project title: Duplicate Question Detection (Vector Search and Semantic Similarity). The objective is to determine whether a new question is semantically similar to one or more questions already stored in a reference collection.

Example: a stored question may be “How to sort a list in Python?” A user may submit “How do I arrange a Python list alphabetically?” The wording is different, but the intended meaning is similar. A simple exact-string comparison would not reliably identify this relationship. A sentence-embedding model represents the meaning of each question as a vector, and a similarity measure compares the vectors.

Problem definition

Input: a new user question and a collection of existing reference questions. Output: a ranked list of reference questions whose similarity score meets a selected threshold. If no question meets the threshold, the program reports that no question reached the threshold.

Project workflow

Stage	What happens
1. Reference data	Keep the existing questions in a Python dictionary.
2. Model loading	Load the pretrained sentence-transformer model once.
3. Offline encoding	Convert the reference questions into vector embeddings.
4. Query encoding	Convert the newly entered question into an embedding.
5. Similarity calculation	Compare the query vector with every reference vector using cosine similarity.
6. Threshold and ranking	Keep candidates at or above the threshold and order them from highest score to lowest.
7. Display	Show the question ID, similarity score, and original question text.

Important distinction

This project is a semantic retrieval / similarity demonstration. It is not a trained binary duplicate-versus-not-duplicate classifier. It compares vectors from a pretrained model and applies a rule-based threshold.

2. Concepts required to understand the project

2.1 Exact matching versus semantic matching

Approach	How it works	Limitation / benefit
Exact text matching	Checks whether two strings are identical or nearly identical character by character.	Misses questions that use different words to express the same meaning.
Keyword matching	Checks whether selected words occur in both questions.	Can miss synonyms and can over-match questions that share words but mean different things.
Semantic similarity	Compares numerical representations produced from the sentence meaning.	Can identify paraphrases, but the score and threshold must be evaluated on representative examples.

2.2 What is a sentence embedding?

A sentence embedding is a fixed-length list of numbers that represents a sentence in a semantic vector space. A pretrained Transformer-based sentence model reads a sentence and converts it into a dense numerical representation. For this project, all-MiniLM-L6-v2 produces 384-dimensional embeddings, as specified in the supplied environment guide.

A simple conceptual example (illustrative only):

```
Question A  ->  [0.12, -0.08, 0.31, ... , 0.04]  (384 numbers)
Question B  ->  [0.11, -0.06, 0.29, ... , 0.02]  (384 numbers)
```

The actual embedding values are generated by the model; they are not manually assigned. Questions with similar meanings are expected to have vectors that are relatively close under the model’s similarity measure.

2.3 What is a vector?

A vector is an ordered collection of numerical values. In this project, one question is represented by 384 values. A collection of eight reference questions therefore produces a matrix-like collection containing eight vectors, each with 384 dimensions. The code stores these embeddings as a tensor so PyTorch-based similarity operations can be applied efficiently.

2.4 What is cosine similarity?

Cosine similarity measures the angle between two vectors. It is calculated as:

```
cosine_similarity(A, B) = (A · B) / (||A|| × ||B||)
```

A dot B is the dot product. ||A|| and ||B|| are the lengths (magnitudes) of the vectors. A cosine similarity closer to 1 indicates that the vectors point in a similar direction. A value closer to 0 indicates little directional similarity; a negative value indicates opposite directions. The range in general mathematics is -1 to +1.

The project normalizes embeddings when they are generated. Normalization makes each vector unit length, which simplifies comparisons and makes cosine similarity equivalent to the dot product for the normalized vectors.

2.5 Threshold

A threshold is the minimum similarity score required for a candidate to be returned. The program uses DEFAULT_THRESHOLD = 0.70. The condition in the code is score >= threshold. A score equal to 0.70 is therefore included.

Threshold choice	Typical effect
Higher threshold	Stricter matching; fewer candidates; some valid paraphrases may be missed.
Lower threshold	More permissive matching; more candidates; unrelated questions may be returned.

Do not interpret similarity as probability

A cosine similarity score of 0.80 does not mean there is an 80% probability that the questions are duplicates. It is a geometric similarity score. A useful threshold must be tested on labelled examples from the intended domain.

2.6 Ranking and top-k

Ranking means ordering candidates from the largest similarity score to the smallest. top_k controls the maximum number of qualifying candidates to display. In the current program, top_k defaults to 5. The threshold is checked before a candidate is appended to the result list.

3. Development environment and folder structure

The supplied environment setup guide uses one shared Python virtual environment for all NLP project folders. This avoids installing large machine-learning packages separately in every project.

3.1 Workspace structure

```
C:\Users\spriy\nlp_projects\
|-- nlp-env\                          # Shared Python virtual environment
|-- duplicate_question_detector\      # Project 1
|   |-- app.py                          # Main command-line program
|   |-- api.py                          # Optional FastAPI endpoint
|   |-- pgvector_schema.sql             # Optional PostgreSQL schema
|   `-- pgvector_store.py               # Optional pgvector integration
|-- content_summarization\            # Project 2
`-- resume_parsing\                   # Project 3
```

The username in the path above reflects the path shown in the supplied VS Code screenshots. On another computer, replace “spriy” with the relevant Windows account folder name.

3.2 Main tools and packages

Tool / package	Purpose in Project 1
Python 3.12	Programming language/runtime used by the shared virtual environment.
VS Code	Editor and terminal used to edit and execute app.py.
nlp-env	Isolated Python environment stored inside the workspace.
sentence-transformers	Loads the pretrained sentence embedding model and encodes sentences.
PyTorch (torch)	Tensor support used by the sentence-transformers library.
Transformers	Underlying model ecosystem used by sentence-transformers.
Hugging Face Hub	Source from which the pretrained model files are downloaded on first use.

3.3 Environment setup commands from the guide

The environment setup document describes the following setup route. In the current session, the shared environment had already been created and the model successfully loaded; these commands are included for students who are reproducing the project from a clean machine.

```
py install 3.12
mkdir nlp_projects
cd nlp_projects
py -3.12 -m venv nlp-env
.\nlp-env\Scripts\Activate.ps1
python -m pip install --upgrade pip setuptools wheel
pip install --default-timeout=1000 --retries 10 --prefer-binary transformers sentence-transformers
```

The broader environment guide also installs optional libraries used by later projects:

```
pip install --default-timeout=1000 --retries 10 --prefer-binary spacy sumy fastapi uvicorn streamlit psycopg2-binary
python -m spacy download en_core_web_sm
```

For this project only

The important package is sentence-transformers. Its dependencies include PyTorch, Transformers, and other numerical libraries. The verified installation in the working environment reported sentence-transformers version 6.3.0.

4. VS Code terminal execution: exact steps followed

4.1 Open the terminal

1. Open the nlp_projects folder in Visual Studio Code.
2. Choose Terminal > New Terminal. The terminal appears in the lower panel.
3. Use the PowerShell terminal. Commands must be typed in the Terminal panel, not inside app.py.

4.2 Navigate to the workspace

```
cd C:\Users\spriy\nlp_projects
```

The prompt should show the workspace folder, similar to:

```
PS C:\Users\spriy\nlp_projects>
```

4.3 Handle the PowerShell execution-policy message (only if it appears)

The first activation attempt was blocked because Windows PowerShell reported that script execution was disabled on the system. The following command was used to allow script execution for the current PowerShell process only:

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
```

This is a temporary, process-scoped setting. It does not permanently change the computer-wide execution policy. After running it, activate the environment.

4.4 Activate the shared virtual environment

When the terminal is at the nlp_projects root, run:

```
.\nlp-env\Scripts\Activate.ps1
```

If the terminal is already inside duplicate_question_detector, the relative path used in the working session is:

```
..\nlp-env\Scripts\Activate.ps1
```

Successful activation is confirmed by the prefix (nlp-env) at the beginning of the terminal prompt:

```
(nlp-env) PS C:\Users\spriy\nlp_projects>
```

Environment check

Always look for (nlp-env) before running the project or installing packages. If it is missing, the terminal may be using a different Python interpreter.

4.5 Enter the project folder

```
cd duplicate_question_detector
```

The expected prompt is similar to:

```
(nlp-env) PS C:\Users\spriy\nlp_projects\duplicate_question_detector>
```

4.6 Verify the package installation

```
python -m pip show sentence-transformers
```

Observed package details from the working session:

```
Name: sentence-transformers
Version: 6.3.0
Location: C:\Users\spriy\nlp_projects\nlp-env\Lib\site-packages
```

This output confirmed that sentence-transformers was installed in the shared nlp-env environment.

4.7 Execute the program

Save app.py with Ctrl+S. From the duplicate_question_detector folder, run:

```
python app.py
```

On its first execution, the program downloads the public pretrained model if it is not already present in the Hugging Face local cache. Once loaded, the program prepares the reference embeddings and displays the input prompt.

5. Complete core program: app.py

This is the complete command-line implementation used for Project 1. Save it as duplicate_question_detector/app.py.

```
"""Duplicate Question Detection using sentence embeddings and cosine similarity.

Run from this folder with:
    python app.py
"""
```

```
from sentence_transformers import SentenceTransformer, util

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
DEFAULT_THRESHOLD = 0.70

# Small classroom dataset. Add more reference questions as needed.
EXISTING_QUESTIONS = {
    1: "How to sort a list in Python?",
    2: "What is the difference between list.sort() and sorted() in Python?",
    3: "How to concatenate strings in Python?",
    4: "How can I merge multiple lists in Python?",
    5: "How do I install a Python package using pip?",
    6: "What is a dictionary in Python and how is it used?",
    7: "How can I read a CSV file using pandas?",
    8: "How do I create a function in Python?",
}


def load_model():
    """Load the sentence-embedding model once."""
    print(f"Loading sentence embedding model: {MODEL_NAME}")
    return SentenceTransformer(MODEL_NAME)


def build_reference_embeddings(model, questions):
    """Encode all reference questions in one batch."""
    question_ids = list(questions.keys())
    question_texts = [questions[q_id] for q_id in question_ids]
    embeddings = model.encode(
        question_texts,
        convert_to_tensor=True,
        normalize_embeddings=True,
        show_progress_bar=False,
    )
    return question_ids, embeddings


def find_duplicates(model, question_ids, reference_embeddings, new_question,
                    threshold=DEFAULT_THRESHOLD, top_k=5):
    """Return reference questions whose cosine similarity meets the threshold."""
    if not new_question or not new_question.strip():
        raise ValueError("Please enter a question before searching.")

    query_embedding = model.encode(
        new_question.strip(), convert_to_tensor=True, normalize_embeddings=True
    )
    similarities = util.cos_sim(query_embedding, reference_embeddings)[0]
    ranked_indices = similarities.argsort(descending=True).tolist()

    matches = []
    for index in ranked_indices:
        score = float(similarities[index].item())
        if score >= threshold:
            q_id = question_ids[index]
            matches.append({
                "question_id": q_id,
                "question_text": EXISTING_QUESTIONS[q_id],
                "similarity": round(score, 4),
            })
        if len(matches) >= top_k:
            break
    return matches


def main():
    model = load_model()
    question_ids, reference_embeddings = build_reference_embeddings(
        model, EXISTING_QUESTIONS
    )
    print("\nReference questions and embeddings are ready.")
    print(f"Number of reference questions: {len(question_ids)}")
    print("\nType a new question. Type 'exit' to stop.\n")

    while True:
        query = input("Enter a question: ").strip()
        if query.lower() in {"exit", "quit"}:
            print("Duplicate Question Detector closed.")
            break
```

```
        if not query:
            print("Please enter a non-empty question.\n")
            continue

        try:
            matches = find_duplicates(
                model, question_ids, reference_embeddings, query,
                threshold=DEFAULT_THRESHOLD,
            )
            if matches:
                print("\nPossible duplicate / similar questions:")
                for match in matches:
                    print(
                        f"  ID {match['question_id']} | "
                        f"Similarity: {match['similarity']:.4f} | "
                        f"{match['question_text']}"
                    )
            else:
                print("\nNo question reached the similarity threshold.")
            print()
        except ValueError as exc:
            print(f"Error: {exc}\n")


if __name__ == "__main__":
    main()
```

6. Code walkthrough

6.1 Import the required classes

```
from sentence_transformers import SentenceTransformer, util
```

SentenceTransformer loads a model designed to create sentence embeddings. util provides similarity utilities, including cosine similarity. This is different from using a general-purpose text-classification pipeline.

6.2 Model name and default threshold

```
MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
DEFAULT_THRESHOLD = 0.70
```

MODEL_NAME identifies the public pretrained model. DEFAULT_THRESHOLD stores the minimum acceptable similarity score. Keeping these as named constants makes it easier to change the model or threshold without searching through the whole program.

6.3 Reference questions

```
EXISTING_QUESTIONS = {
    1: "How to sort a list in Python?",
    2: "What is the difference between list.sort() and sorted() in Python?",
    3: "How to concatenate strings in Python?",
    4: "How can I merge multiple lists in Python?",
    5: "How do I install a Python package using pip?",
    6: "What is a dictionary in Python and how is it used?",
    7: "How can I read a CSV file using pandas?",
    8: "How do I create a function in Python?",
}
```

A Python dictionary is used so each question has a stable ID. The ID is displayed with its matching result. A larger system could load this reference collection from a CSV file, database, learning-management system or question bank.

6.4 Load the model once

```
def load_model():
    print(f"Loading sentence embedding model: {MODEL_NAME}")
    return SentenceTransformer(MODEL_NAME)
```

The function loads the pretrained model and returns it. The program calls it once near the start of main(). Loading once avoids repeatedly reloading the model for every user query.

6.5 Convert reference questions into embeddings

```
def build_reference_embeddings(model, questions):
    question_ids = list(questions.keys())
    question_texts = [questions[q_id] for q_id in question_ids]
    embeddings = model.encode(
        question_texts,
        convert_to_tensor=True,
        normalize_embeddings=True,
        show_progress_bar=False,
    )
    return question_ids, embeddings
```

The function separates IDs from question text, passes all reference texts to model.encode(), and returns both the ordered IDs and the vectors. Encoding all references together is an offline preparation step. convert_to_tensor=True requests a PyTorch tensor. normalize_embeddings=True normalizes each vector. show_progress_bar=False hides the model's internal progress bar for this small classroom dataset.

Why create reference embeddings first?

The stored questions do not change for every user query. Their vectors can be prepared once and reused. For each new question, only the query vector needs to be generated.

6.6 Find similar questions

```
def find_duplicates(model, question_ids, reference_embeddings, new_question,
                    threshold=DEFAULT_THRESHOLD, top_k=5):
    if not new_question or not new_question.strip():
        raise ValueError("Please enter a question before searching.")

    query_embedding = model.encode(
        new_question.strip(), convert_to_tensor=True, normalize_embeddings=True
    )
    similarities = util.cos_sim(query_embedding, reference_embeddings)[0]
    ranked_indices = similarities.argsort(descending=True).tolist()

    matches = []
    for index in ranked_indices:
        score = float(similarities[index].item())
        if score >= threshold:
            q_id = question_ids[index]
            matches.append({
                "question_id": q_id,
                "question_text": EXISTING_QUESTIONS[q_id],
                "similarity": round(score, 4),
            })
        if len(matches) >= top_k:
            break
    return matches
```

The function first rejects blank input. It then encodes the query in the same vector space as the stored questions. util.cos_sim() calculates one similarity score for each reference vector. argsort(descending=True) arranges vector positions in descending score order. The loop converts each score to a regular Python float, checks the threshold, and appends qualifying candidates. round(score, 4) formats the score to four decimal places. The function returns no more than top_k matching questions.

6.7 Main interactive loop

```
def main():
    model = load_model()
    question_ids, reference_embeddings = build_reference_embeddings(
        model, EXISTING_QUESTIONS
    )
    print("\nReference questions and embeddings are ready.")
    print(f"Number of reference questions: {len(question_ids)}")
    print("\nType a new question. Type 'exit' to stop.\n")

    while True:
```

```
        query = input("Enter a question: ").strip()
        if query.lower() in {"exit", "quit"}:
            print("Duplicate Question Detector closed.")
            break
        if not query:
            print("Please enter a non-empty question.\n")
            continue

        try:
            matches = find_duplicates(
                model, question_ids, reference_embeddings, query,
                threshold=DEFAULT_THRESHOLD,
            )
            if matches:
                print("\nPossible duplicate / similar questions:")
                for match in matches:
                    print(
                        f"  ID {match['question_id']} | "
                        f"Similarity: {match['similarity']:.4f} | "
                        f"{match['question_text']}"
                    )
            else:
                print("\nNo question reached the similarity threshold.")
            print()
        except ValueError as exc:
            print(f"Error: {exc}\n")
```

while True keeps asking for questions until the user types exit or quit. strip() removes leading and trailing spaces. The program checks for empty input, calls the matching function, and prints the candidate ID, score and original text. If there are no matches, it prints a threshold message. A ValueError is caught and displayed without crashing the application.

6.8 Python entry-point guard

```
if __name__ == "__main__":
    main()
```

This guard calls main() only when app.py is run directly. It prevents the interactive loop from automatically starting if the file is imported by another Python module.

7. Testing the detector and interpreting output

7.1 The observed successful model-loading checkpoint

The terminal output shared during the working session included these important lines:

```
Loading sentence embedding model: sentence-transformers/all-MiniLM-L6-v2

Reference questions and embeddings are ready.
Number of reference questions: 8

Type a new question. Type 'exit' to stop.

Enter a question:
```

This confirms that the model loaded and that eight reference questions were encoded. The input prompt means the application was waiting for the first query. It does not, by itself, confirm any particular duplicate match or similarity score.

7.2 Suggested classroom test cases

Test	Enter this question	What to look for
1. Similar wording	How do I sort a list alphabetically in Python?	Inspect whether the reference question about sorting a list appears near the top.
2. Paraphrase	How can I combine several Python lists?	Inspect whether the reference question about merging multiple lists is returned.

3. Related concept	When should I use sorted() instead of list.sort()?	Inspect whether the difference between list.sort() and sorted() is ranked.
4. Unrelated subject	What is photosynthesis?	Check whether any reference question incorrectly passes the threshold.
5. Blank input	Press Enter without typing a question.	The program should ask for a non-empty question.
6. Exit	exit	The program should close the detector loop.

The “What to look for” column describes an evaluation expectation, not a guaranteed exact score. Results depend on the model, the wording and the threshold. Record the actual output during the practical session.

7.3 How to read the result

A result line has the following format:

```
ID 1 | Similarity: 0.xxxx | How to sort a list in Python?
```

ID identifies the stored reference question. Similarity is the cosine similarity rounded to four decimal places. The text is the original reference question. A candidate is displayed only when its score is greater than or equal to 0.70 in the current configuration.

7.4 Evaluate threshold quality

For a more rigorous classroom experiment, label a set of query/reference pairs as “duplicate” or “not duplicate”. Run the detector at several thresholds (for example, 0.60, 0.70, 0.80 and 0.90) and record the true positives, false positives, true negatives and false negatives. This helps explain why a threshold must be selected using the task’s data rather than treated as a universal constant.

8. Troubleshooting log from the actual setup session

8.1 Error: ModuleNotFoundError: No module named sentence_transformers

Observed cause: app.py was run from a terminal whose prompt did not show (nlp-env). Although the package was installed in the shared environment, the active terminal was using a different Python interpreter.

Resolution: activate the environment, verify the package using python -m pip show, and run the program from that same terminal.

```
..\nlp-env\Scripts\Activate.ps1
python -m pip show sentence-transformers
python app.py
```

8.2 PowerShell says running scripts is disabled

Observed message: Activate.ps1 cannot be loaded because running scripts is disabled on this system.

Resolution used in the session: temporarily allow scripts in the current PowerShell process and activate the environment again.

```
Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
..\nlp-env\Scripts\Activate.ps1
```

If the terminal is at the workspace root rather than inside the project folder, use .\nlp-env\Scripts\Activate.ps1 instead of the relative path shown above.

8.3 Warning: unauthenticated requests to Hugging Face Hub

Meaning: the computer downloaded a public model without an authenticated Hugging Face token. This warning is not a model-loading failure. A token can provide higher request limits, but authentication is not required for this public classroom model under ordinary conditions.

8.4 Warning: Windows cache does not support symlinks

Meaning: the Hugging Face cache cannot use symbolic links on this Windows configuration. Files are still cached using a fallback method, which may use more disk space. The warning does not prevent model loading. Developer Mode or administrator execution can enable symlinks, but neither is required to complete this local demonstration.

8.5 Initial code used a classification pipeline

The first app.py version shown in the screenshot used a transformers text-classification pipeline with the all-MiniLM-L6-v2 model. This was the wrong task interface for the intended project. A sentence embedding model should be loaded through SentenceTransformer and used to encode sentences. The missing classifier.weight and classifier.bias warning in the initial attempt referred to classification-layer parameters that were not part of the sentence-embedding checkpoint. The corrected code no longer uses a text-classification head.

```
from sentence_transformers import SentenceTransformer, util
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
```

8.6 Package installed but still not found

Use python -m pip rather than pip by itself so that the package check or installation is associated with the Python interpreter currently selected in the terminal.

```
python -m pip show sentence-transformers
python -c "import sentence_transformers; print(sentence_transformers.__version__)"
```

8.7 Model download is interrupted or slow

The first run may need to download model configuration and weight files. Retry when the network is stable. Subsequent runs generally reuse the local Hugging Face cache. A download progress bar is expected during first-time model setup.

9. Classroom exercises, evaluation and viva questions

9.1 Hands-on exercises

1. Add five new reference questions to EXISTING_QUESTIONS. Give every question a unique integer ID.
2. Test a direct paraphrase and a question on a different topic. Record the returned IDs and scores.
3. Change DEFAULT_THRESHOLD from 0.70 to 0.80. Repeat the same tests and compare the number of results.
4. Change top_k from 5 to 2 in the call to find_duplicates(). Observe the maximum number of returned candidates.
5. Submit a blank input and then type exit. Explain the two different control-flow paths.
6. Discuss why a high similarity score should not automatically be treated as proof that two questions are identical.

9.2 Suggested observation table

Query entered	Top matched ID	Similarity score	Threshold	Class label after human review
			0.70	
			0.70	
			0.70	
			0.70	

9.3 Viva questions

Question	Expected answer
What is an embedding?	A numerical vector representation of a text item.
Why use a sentence-transformer model?	It is designed to produce sentence-level embeddings that can be compared by similarity.
What does all-MiniLM-L6-v2 do in this project?	It converts each question into a 384-dimensional sentence embedding.
What is cosine similarity?	A measure based on the cosine of the angle between two vectors.
Why are reference questions encoded before user input?	Their embeddings can be reused; only the incoming question needs to be encoded for each search.
What does threshold = 0.70 mean?	Only candidates with similarity scores greater than or equal to 0.70 are returned.
Does a score of 0.80 mean an 80% chance of duplication?	No. It is a similarity score, not a calibrated probability.
What is top_k?	The maximum number of qualifying candidates returned.
Why activate nlp-env?	To ensure the correct Python interpreter and installed packages are used.
What is the difference between this program and a classifier?	This program retrieves and ranks semantically similar questions using embeddings and a threshold; it does not train a duplicate/non-duplicate classifier.

10. Optional extension A: expose the detector using FastAPI

The local command-line application should be working before this extension is attempted. FastAPI provides a web API endpoint so another program or user interface can submit a question and receive JSON results.

10.1 Run the API

Open a terminal in duplicate_question_detector with (nlp-env) active. Run:

```
uvicorn api:app --reload
```

Open this address in a web browser:

```
http://127.0.0.1:8000/docs
```

In the interactive API page, expand POST /detect, choose Try it out, and submit:

```
{
  "question": "How can I arrange a Python list in ascending order?",
  "threshold": 0.70,
  "top_k": 5
}
```

The endpoint returns the input question, threshold, candidate matches, and a match_found flag. Stop the server using Ctrl+C in its terminal.

10.2 Complete API code

```
"""Optional FastAPI interface for the duplicate-question detector.

Run from this folder with:
    uvicorn api:app --reload
Then open http://127.0.0.1:8000/docs
"""

from fastapi import FastAPI, HTTPException
from pydantic import BaseModel, Field
from sentence_transformers import SentenceTransformer, util

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
QUESTIONS = {
    1: "How to sort a list in Python?",
    2: "What is the difference between list.sort() and sorted() in Python?",
    3: "How to concatenate strings in Python?",
    4: "How can I merge multiple lists in Python?",
    5: "How do I install a Python package using pip?",
```

```
    6: "What is a dictionary in Python and how is it used?",
    7: "How can I read a CSV file using pandas?",
    8: "How do I create a function in Python?",
}

app = FastAPI(title="Duplicate Question Detector", version="1.0")
model = SentenceTransformer(MODEL_NAME)
question_ids = list(QUESTIONS.keys())
reference_embeddings = model.encode(
    [QUESTIONS[q_id] for q_id in question_ids],
    convert_to_tensor=True,
    normalize_embeddings=True,
)


class QueryRequest(BaseModel):
    question: str = Field(min_length=3, description="Question to check")
    threshold: float = Field(default=0.70, ge=-1.0, le=1.0)
    top_k: int = Field(default=5, ge=1, le=20)


@app.get("/")
def home():
    return {"message": "Duplicate Question Detector API is running."}


@app.post("/detect")
def detect_duplicates(request: QueryRequest):
    query = request.question.strip()
    if not query:
        raise HTTPException(status_code=400, detail="Question cannot be blank.")

    query_embedding = model.encode(
        query, convert_to_tensor=True, normalize_embeddings=True
    )
    scores = util.cos_sim(query_embedding, reference_embeddings)[0]
    order = scores.argsort(descending=True).tolist()
    results = []
    for index in order:
        score = float(scores[index].item())
        if score >= request.threshold:
            q_id = question_ids[index]
            results.append({
                "question_id": q_id,
                "question_text": QUESTIONS[q_id],
                "similarity": round(score, 4),
            })
        if len(results) >= request.top_k:
            break
    return {"input_question": query, "threshold": request.threshold,
            "matches": results, "match_found": bool(results)}
```

11. Optional extension B: PostgreSQL and pgvector

This extension stores question embeddings in PostgreSQL and performs nearest-neighbour search in the database. It is not required for the current in-memory command-line application. It requires a running PostgreSQL server with the pgvector server extension installed.

11.1 Prerequisites

1. Install and start PostgreSQL.
2. Install the pgvector extension for the PostgreSQL server and confirm that the server can run CREATE EXTENSION vector.
3. Install the Python adapter packages inside nlp-env.
4. Create a database and keep the connection credentials private.

```
python -m pip install psycopg2-binary pgvector
$env:DATABASE_URL = "postgresql://postgres:YOUR_PASSWORD@localhost:5432/nlpdb"
```

Credential safety

Replace the example connection string with your own local database settings. Do not publish a real password in app.py, a shared document, source control, or screenshots.

11.2 Database schema

The schema creates a table with a question ID, question text and a 384-dimensional vector column. It also defines an HNSW index using cosine distance for approximate nearest-neighbour queries, provided the installed pgvector server version supports HNSW.

```
-- Optional database extension exercise for PostgreSQL with pgvector installed.
-- all-MiniLM-L6-v2 produces 384-dimensional embeddings.

CREATE EXTENSION IF NOT EXISTS vector;

CREATE TABLE IF NOT EXISTS questions (
    question_id BIGSERIAL PRIMARY KEY,
    question_text TEXT NOT NULL UNIQUE,
    embedding VECTOR(384) NOT NULL
);

-- Optional approximate-nearest-neighbour index (requires a pgvector version
-- that supports HNSW indexes).
CREATE INDEX IF NOT EXISTS questions_embedding_hnsw_idx
ON questions USING hnsw (embedding vector_cosine_ops);

-- Example nearest-neighbour query. Bind the input vector as a parameter from
-- Python rather than typing the placeholder literally in pgAdmin.
-- SELECT question_id, question_text,
--        1 - (embedding <=> '[...]'::vector) AS cosine_similarity
-- FROM questions
-- ORDER BY embedding <=> '[...]'::vector
-- LIMIT 5;
```

11.3 Run the database demonstration

```
cd C:\Users\spriy\nlp_projects\duplicate_question_detector
python pgvector_store.py
```

The Python program connects through DATABASE_URL, creates the table if needed, inserts sample questions and searches for the nearest questions. If DATABASE_URL is not set or the database server is not running, the optional demonstration cannot connect. This does not affect the local app.py program.

11.4 Complete Python database integration code

```
"""Optional PostgreSQL + pgvector persistence for MiniLM question embeddings.

Prerequisites:
  1. PostgreSQL server with the pgvector extension installed.
  2. pip install psycopg2-binary pgvector
  3. Set DATABASE_URL in PowerShell, for example:
     $env:DATABASE_URL = "postgresql://postgres:YOUR_PASSWORD@localhost:5432/nlpdb"

Run from this folder:
  python pgvector_store.py
"""

import os
import numpy as np
import psycopg2
from pgvector.psycopg2 import register_vector
from sentence_transformers import SentenceTransformer

MODEL_NAME = "sentence-transformers/all-MiniLM-L6-v2"
SAMPLE_QUESTIONS = [
    "How to sort a list in Python?",
    "What is the difference between list.sort() and sorted() in Python?",
    "How to concatenate strings in Python?",
    "How can I merge multiple lists in Python?",
]
```

```
def connect_database():
    database_url = os.getenv("DATABASE_URL")
    if not database_url:
        raise RuntimeError("Set DATABASE_URL before running this optional database exercise.")
    connection = psycopg2.connect(database_url)
    register_vector(connection)
    return connection


def initialize_table(connection):
    with connection.cursor() as cursor:
        cursor.execute("CREATE EXTENSION IF NOT EXISTS vector;")
        cursor.execute("""
            CREATE TABLE IF NOT EXISTS questions (
                question_id BIGSERIAL PRIMARY KEY,
                question_text TEXT NOT NULL UNIQUE,
                embedding VECTOR(384) NOT NULL
            );
        """)
    connection.commit()


def store_questions(connection, model, questions):
    for question in questions:
        vector = model.encode(question, convert_to_numpy=True, normalize_embeddings=True)
        vector = np.asarray(vector, dtype=np.float32)
        with connection.cursor() as cursor:
            cursor.execute(
                """INSERT INTO questions (question_text, embedding)
                   VALUES (%s, %s)
                   ON CONFLICT (question_text)
                   DO UPDATE SET embedding = EXCLUDED.embedding;""",
                (question, vector),
            )
    connection.commit()


def search_similar_questions(connection, model, query, top_k=5):
    query_vector = model.encode(query, convert_to_numpy=True, normalize_embeddings=True)
    query_vector = np.asarray(query_vector, dtype=np.float32)
    with connection.cursor() as cursor:
        cursor.execute(
            """SELECT question_id, question_text,
                      1 - (embedding <=> %s) AS similarity
               FROM questions
               ORDER BY embedding <=> %s
               LIMIT %s;""",
            (query_vector, query_vector, top_k),
        )
        return cursor.fetchall()


def main():
    print("Loading MiniLM sentence-embedding model...")
    model = SentenceTransformer(MODEL_NAME)
    connection = connect_database()
    try:
        initialize_table(connection)
        store_questions(connection, model, SAMPLE_QUESTIONS)
        query = input("Enter a question to search in PostgreSQL (or press Enter for a demo query): ").strip()
        if not query:
            query = "How do I arrange a Python list?"
        print("\nNearest questions from pgvector:")
        for question_id, question_text, similarity in search_similar_questions(connection, model, query):
            print(f"ID {question_id} | similarity={float(similarity):.4f} | {question_text}")
    finally:
        connection.close()


if __name__ == "__main__":
    main()
```

12. Teaching notes and implementation limitations

12.1 Suggested teaching sequence

Teaching phase	Suggested activity

Concept introduction	Explain exact matching, semantic matching, embeddings and vector similarity.
Environment check	Ask students to identify the active virtual environment and verify the package.
Model loading	Run the program and explain the first model download.
Reference encoding	Explain that eight stored questions are converted into vectors once.
Live testing	Enter paraphrases and unrelated questions. Discuss ranking and threshold behaviour.
Parameter study	Change the threshold and top_k. Compare the results.
Advanced extension	Demonstrate FastAPI and discuss pgvector as a persistence and scaling option.

12.2 Known limitations

- The classroom dataset contains only eight reference questions. It is intended to demonstrate the method, not provide production-scale coverage.
- The threshold of 0.70 is an initial teaching value. It must be evaluated against representative labelled examples.
- Semantically related questions are not always duplicates. Human review may be needed before merging or rejecting questions.
- The program currently uses an in-memory dictionary. Reference questions are lost when the program closes unless they are stored in a file or database.
- The model is pretrained and is not fine-tuned on a domain-specific duplicate-question dataset in this implementation.
- The FastAPI and PostgreSQL integrations are optional deployment extensions and were not part of the successful local model-loading checkpoint documented in the terminal log.

12.3 Suggested assessment rubric

Assessment item	Marks (suggested)
Environment activation and package verification	10
Correct loading of sentence-transformer model	15
Correct construction of reference embeddings	20
Cosine similarity and ranking explanation	20
Testing with paraphrased and unrelated questions	15
Threshold experiment and observation table	10
Code explanation and viva	10
Total	100

13. Quick reference: commands to keep handy

Task	PowerShell command
Go to workspace	cd C:\Users\spriy\nlp_projects
Temporary PowerShell script permission (only if blocked)	Set-ExecutionPolicy -Scope Process -ExecutionPolicy Bypass -Force
Activate from workspace root	.\nlp-env\Scripts\Activate.ps1
Activate from project folder	..\nlp-env\Scripts\Activate.ps1
Enter Project 1 folder	cd duplicate_question_detector
Verify package	python -m pip show sentence-transformers
Run core program	python app.py
Run API extension	uvicorn api:app --reload

Stop a running app/server	Ctrl+C

Source alignment and notes

This manual follows the supplied “NLP Projects” PDF for the project objective and the duplicate-question detection workflow, and the supplied “NLP Environment Setup & Duplicate Question Detector Guide (Revised)” PDF for the shared nlp-env workspace and the MiniLM-based local implementation. The implementation shown in this manual is the app.py file in the accompanying project package.

The main project PDF also mentions all-mpnet-base-v2 in an introductory section, while the environment guide and the current implementation use all-MiniLM-L6-v2. These notes retain all-MiniLM-L6-v2 because it is the model used by the working code and the verified environment guide. The main project brief includes FastAPI and pgvector as deployment extensions; they are presented here as optional stages after the local detector works.

Checkpoint for the next learning session

At the end of this session, the terminal had displayed “Reference questions and embeddings are ready”, “Number of reference questions: 8”, and “Enter a question:”. The next practical action is to enter a test query, observe the actual ranked output, and record the real similarity scores in the observation table.